

1. Select the 'Create new project' button.
2. Select 'Manually' for generating the project.
3. A new page loads, where we must assign a 'Project Key' and a 'Display Name' for the project and confirm it by pressing the 'Set Up' button. These need to be unique.
4. Now, we are redirected to another page where we must generate a token. Under 'Provide a Token', select 'Generate a Token' to generate it, or enter the name of an existing token if you wish to reuse the token. This token is used to generate the command for running the analysis on CLI.
5. Select the major language and go along with the specified instructions to examine your project. As we are using Maven or Gradle for our project, the scanner has been downloaded by default. SonarQube generates a command, as shown below:

```
mvn sonar:sonar \
-Dsonar.projectKey=cms-java-sast \
-Dsonar.host.url=http://localhost:9000 \
-Dsonar.login=c88f4b568cc5f669d48b9ff0f6
acca139dd3a6a5
```

This command is different for different tokens and project names. Run the command generated by SonarQube for testing your project.

6. Now, go to the project folder and run the above command there. On successful analysis, *Build Success* will look as shown in Figure 7.

After successfully passing inspection of your code, you will see your foremost analysis on SonarQube (localhost: 9000), as manifested in Figure 8. It is evident from Figure 8 that the analysis was successful and there are three bugs, zero vulnerabilities, and eleven security hotspots. Code smells and technical debt time are also shown in the report.

On switching to the 'Issues' tab in the dashboard, the screen shown by Figure 9 comes up. All the issues of different

**snyk**

Snyk test report

Scanned the following path:  
• /home/htriyal/jenkins/workspace/SAST-SNYK (maven)

7 known vulnerabilities | 7 vulnerable dependency paths | 78 dependencies

Project: com.spe ClassroomManagementSystem Path: /home/htriyal/jenkins/workspace/SAST-SNYK

Package Manager: maven Manifest: pom.xml

**HIGH SEVERITY**

**SQL Injection**

- Package Manager: maven
- Vulnerable module: org.hibernate:hibernate-core
- Introduced through: com.spe:ClassroomManagementSystem@0.0.1-SNAPSHOT, org.springframework.boot:spring-boot-starter-data-jpa@2.2.6.RELEASE and others

**Detailed paths**

- Introduced through: com.spe:ClassroomManagementSystem@0.0.1-SNAPSHOT > org.springframework.boot:spring-boot-starter-data-jpa@2.2.6.RELEASE > org.hibernate:hibernate-core@5.4.12.Final

**Overview**

[org.hibernate:hibernate-core](#) is a library providing ObjectRelational Mapping (ORM) support to applications, libraries, and frameworks.

Affected versions of this package are vulnerable to SQL Injection. A SQL injection in the implementation of the JPA Criteria API can permit unauthorized literals when a literal is used in the SELECT or GROUP BY parts of the query. This flaw could allow an attacker to access unauthorized information or possibly conduct further attacks.

**Remediation**

Upgrade [org.hibernate:hibernate-core](#) to version 5.3.18.Final, 5.4.18.Final or higher.

**References**

- [GitHub Pull Request](#)
- [Jira Ticket](#)

[More about this vulnerability](#)

Figure 6: Snyk test report

severity are listed. Navigation to other properties like resolution, status, etc, is also possible from the left panel.

## Analysing a Python project

The Python project that we will be using is available on GitHub at <http://github.com/RahulSapparu/iitbfeedback>. For the Python project, we must download the sonar-scanner at <https://docs.sonarqube.org/latest/analysis/scan/sonarscanner/>. For Python projects, no distinct scanner is available. Therefore, we must install 'SonarScanner', which solves this issue as it is used when there is no distinct scanner at hand for your build system.

1. Go to your project's root directory

and create a file named *sonar-project.properties*. This will serve as a configuration file for the project; the contents of the file are given below:

```
# must be unique in a given SonarQube instance
sonar.projectKey=my:project
# --- optional properties ---
# defaults to project key
#sonar.projectName=My project
# defaults to 'not provided'
#sonar.projectVersion=1.0
# Path is relative to the sonar-project.properties file. Defaults to .
#sonar.sources=.
# Encoding of the source code. Default is default system encoding
#sonar.sourceEncoding=UTF-8
```